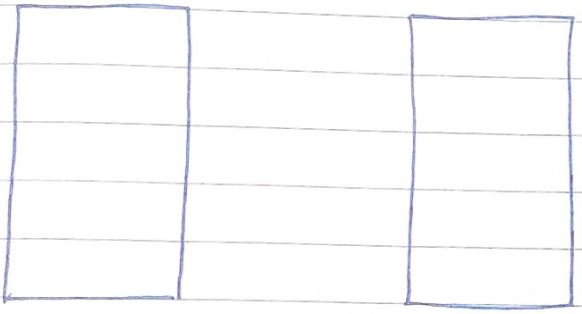
Lab-1 State Space And Combinatorial Explosion

Jug X holds 4 Liters. Jug Y holds 3 Litres. Neither has markings. Both start empty. Water is unlimited.
Six Operations are allowed to perform on the jugs:

1. Fill X - X becomes full i.e 4
2. Fill Y - Y becomes full i.e 3
3. Empty X - X becomes empty i.e 0
4. Empty Y - Y becomes empty i.e 0
5. Pour X → Y - Pour water until X becomes empty or Y becomes full
6. Pour Y → X - Pour water until Y becomes empty or X becomes full



State
(0, 0)

X = 0
Capacity 4

Y = 0
Capacity 3

Step 1: Define the state

In the water jug problem, a state is simply (x, y), where x and y represents the amount of water in the two jugs. This is an important decision in designing i.e. defining the state as it forms the foundation of the program.

Step 2: Define successors() Function

```
def successors (state):
    x, y = state
```

what the function does: Generates all possible next states from the current state

x, y = state: This unpacks the python tuple. For example if state = (4, 1) then x = 4 and y = 1.

Step 3: Work out how much water actually moves

pour_x_to_y = min (x, 3-y)

pour_y_to_x = min (4-x, y)

X → Y:

While pouring from X to Y, X can only give the water it currently has. Y can only receive water until it becomes full.

So we take the minimum of

- water available in X
- empty space available in Y

Ex: state = (4,1)

⇒ X = 4 Y = 1

⇒ remaining space in Y = 3-1 = 2L

⇒ pour_x_to_y = min (4, 2) = 2

⇒ only 2L can be poured

Step 4: Return all possible moves

The function returns a plain list of six new states. It does not check whether these states are new, are they leagal or are there any duplicates.

return [(4, y), (x, 3), (0, y), (x, 0),
(x - pour_x_to_y, y + pour_x_to_y),
(x + pour_y_to_x, y - pour_y_to_x)]

Step 5: Declare the variables for search

start = (0,0)

Initial state where both the jugs are empty i.e. X=0, Y=0

seen = {start}

seen is a set holding every state ever found. This prevents revisiting the same state and avoids infinite loops.

frontier = [start]

frontier is a list that holds all the states that need to be exepanded at the current level.

level = 0

This counts how deep we are. How many moves have been made from the starting node.

Step 6: Start the level loop

while frontier and level < 8:

    new = []

The loop continues while:

-> there are states to explore, and

-> the search depth is less than 8.

new = []: collects all the states in this level only. Empties for every level

Step 7: Expand each state in current frontier & generate successor state

for s in frontier:

    for ns in successors(s):

The outer loop traverses each state in the current frontier and the inner loop walks each of the six states that one move can reach.

Together they generate every state reachable in exactly one more move than the current level.

Step 8: Checking the State

if ns not in seen:

    seen.add(ns)

    new.append(ns)

Only process a state if it has never been seen before. If ns is already present in seen ignore it otherwise add the state to the seen set and append to the new list more states are added as they are discovered.

Step 9: Check for the goal

```
if ns[0] == 2:
    print ("Goal : 2L in X at level", level+1, )
```

ns[0] is the first element of the tuple which is jug X. So ns[0] == 2 means it is asking whether there are 2L in the jug X.

If yes we print the level +1 since it has not been incremented yet in this part of the loop.

Step 10: Move on to the next level

```
level += 1
print ("Level", level, "→", len(new), "new states:", sorted(new))
frontier = new
```

We are moving on to the next level of the search. Whatever is discovered in this round will be expanded in the next round and everything found earlier will not be revisited again since they are already in seen.

Step 11: Print total no of States

```
print ("Total states found:", len(seen))
```

len(seen) is the size of the entire reachable state space. This tells us how many distinct water configurations were explored.